

SOFTWARE REQUIREMENTS SPECIFICATION

Airbag ECU — Dual Arduino Due Redundant System

Document No.	SRS-AIRBAG-SW-001
Revision	2.0
Date	2026-06-10
Status	DRAFT
ASIL / SIL	ASIL D / SIL 3
Target HW	Arduino Due (SAM3X8E ARM Cortex-M3) × 2
Standards	ISO 26262:2018 / IEC 61508:2010
Based On	FSRS-AIRBAG-ECU-001 Rev 2.1 SC-AIRBAG-ECU-001 Rev 2.0 HARA-AIRBAG-ECU-001 Rev 2.1
Author	Om Naik, Mrunal Ghotekar

1. Purpose and Scope

This Software Requirements Specification (SRS) defines all software-level requirements for the Airbag ECU embedded software running on two Arduino Due microcontroller boards Channel A (Due 1, Master) and Channel B (Due 2, Slave). It is derived from and fully traceable to the Functional Safety Requirements Specification (FSRS-AIRBAG-ECU-001 Rev 2.1), the Safety Concept Document (SC-AIRBAG-ECU-001 Rev 2.0), and the Hazard and Risk Analysis (HARA-AIRBAG-ECU-001 Rev 2.1).

The software is classified ASIL D per ISO 26262:2018 and SIL 3 per IEC 61508:2010. All requirements defined in this document are mandatory unless explicitly marked optional. No requirement may be waived without a formal change impact assessment and re-evaluation of the HARA and ASIL classification.

2. Document Information

2.1 Revision History

Rev.	Date	Author	Description
2.0	2026-06-10	Om Naik, Mrunal Ghotekar	Second release

2.2 Reference Documents

Document Number	Title	Revision
HARA-AIRBAG-ECU-001	Hazard and Risk Analysis	Rev 2.1
FSRS-AIRBAG-ECU-001	Functional Safety Requirements Specification	Rev 2.1
SC-AIRBAG-ECU-001	Safety Concept Document	Rev 2.0
HADS-AIRBAG-ECU-001	Hardware Architecture and Design Specification	Rev 1.0
ISO 26262:2018	Road Vehicles — Functional Safety (Parts 3, 5, 6)	2018 Edition
IEC 61508:2010	Functional Safety of E/E/PE Systems (Parts 2, 3)	2010 Edition

2.3 Abbreviations

Term	Definition
ADC	Analog-to-Digital Converter
ASIL	Automotive Safety Integrity Level (ISO 26262)
CAN	Controller Area Network
DWI	Diagnostic Warning Indicator

ECU	Electronic Control Unit
FSR	Functional Safety Requirement (from FSRS)
GPIO	General Purpose Input/Output
OMS	Original Manufacturer System (laboratory monitoring PC)
POST	Power-On Self-Test
SIL	Safety Integrity Level (IEC 61508)
SRS	Software Requirements Specification (this document)
SWR	Software Requirement (individual requirement in this document)
WDT	Hardware Watchdog Timer
2oo2	Two-out-of-Two voting architecture
CRC	Cyclic Redundancy Check
DLC	Data Length Code (CAN frame field)

3. System Overview

3.1 Hardware Platform

The software runs on two Arduino Due boards, each featuring the Atmel SAM3X8E ARM Cortex-M3 microcontroller running at 84 MHz. Each board independently reads all three sensor inputs, executes the crash detection algorithm, and drives its own AND gate input pins. The hardware DM74LS08N AND gate enforces the physical 2oo2 deployment constraint, both boards must simultaneously assert their output HIGH for the deployment LED to activate. CAN1 provides inter-board heartbeat and status communication. CAN0 on Due 1 connects to a WaveShare SN65HVD230 CAN transceiver for OMS lab PC communication at 500 kbit/s.

3.2 Board Roles and Responsibilities

Board	Role	CAN Interfaces	Key Software Responsibilities
Arduino Due 1 (Channel A)	Master	CAN0 → OMS PC (monitoring) CAN1 ↔ Channel B (inter-channel)	Sensor acquisition, crash detection, deployment output drive, inter-channel CAN supervision, OMS PC CAN transmission, DWI control
Arduino Due 2 (Channel B)	Slave	CAN1 ↔ Channel A (inter-channel)	Sensor acquisition, crash detection, deployment output drive, inter-channel CAN status transmission to Channel A, DWI control

3.3 Software Safety Classification

Software Function	ISO 26262	IEC 61508	Governing FSR
POST self-tests, crash detection, deployment logic, watchdog	ASIL D	SIL 3	FSR-03, FSR-07, FSR-17, FSR-22
Sensor acquisition and physical conversion	ASIL D	SIL 3	FSR-01, FSR-02
Baby seat detection and passenger suppression	ASIL D	SIL 3	FSR-09, FSR-10, FSR-11
Diagnostic monitoring and safe-state transitions	ASIL D	SIL 3	FSR-18, FSR-20
OMS PC CAN communication (monitoring only)	ASIL B	SIL 2	FSR-13, FSR-16
Diagnostic Warning Indicator (DWI) lamp	ASIL A	SIL 1	FSR-19
Serial monitor diagnostic output	ASIL A	SIL 1	FSR-21

4. Software Module Definitions

The following table defines every software module constituting the complete embedded software. Each module maps to one or more FSR requirements and is assigned an ASIL classification.

Module	Function	Runs On	Cycle	FSR	ASIL
cpu_test()	Arithmetic self-test of CPU core	Both	Once at boot	FSR-17	ASIL C
ram_test()	Checkerboard write/read pattern test of RAM	Both	Once at boot	FSR-17, FSR-22	ASIL D
adc_test()	Verify ADC returns a value in valid range 0–4095	Both	Once at boot	FSR-17	ASIL C
watchdog_setup()	Configure SAM3X8E hardware WDT with 10 ms timeout	Both	Once at boot	FSR-22	ASIL D
watchdog_feed()	Kick hardware WDT every loop	Both	< 10 ms	FSR-22	ASIL D
updateBabySeat()	Read D11, apply 10 ms debounce, set stable babySeat state	Both	Every loop	FSR-09, 10, 11	ASIL D

toPhys()	Scale 12-bit ADC reading to physical unit	Both	Every loop	FSR-01, 02	ASIL D
Sensor read block	Read A0/A1/A2, convert to g / deg/s / kPa	Both	Every loop	FSR-01, 02	ASIL D
Threshold check block	Compare physical values against THRESH_x constants	Both	Every loop	FSR-03, 04	ASIL D
crashLocal decision	OR of accelEx, gyroEx, pressEx flags	Both	Every loop	FSR-03	ASIL D
driveAirbag1/2 logic	Set D8/D9 HIGH only when crash confirmed and no fault	Both	Every loop	FSR-03, 04, 08	ASIL D
DWI logic	Drive D10 HIGH on any local or remote fault flag	Both	Every loop	FSR-19, 20	ASIL A/D
enterSafeState logic	Force D8/D9 LOW when system_fault TRUE	Both	Every loop	FSR-20	ASIL D
CAN heartbeat TX	Transmit heartbeat every 500 ms via CAN bus	Both	500 ms	FSR-07, 08	ASIL D
CAN heartbeat RX	Receive and parse peer heartbeat messages from peer channel	Both	Every loop	FSR-07, 08	ASIL D
CAN timeout check	Set fault_peerChannel if no heartbeat received within timeout period	Both	Every loop	FSR-15	ASIL D
parseCAN() / status TX	Due 2 sends crash/baby/fault; Due 1 parses remote state	Due 1 + Due 2	100 ms	FSR-07	ASIL D
crc16()	Calculate CRC-16 (poly 0x1021, seed 0xFFFF) over 6 CAN bytes	Due 1 only	On CAN TX	FSR-13	ASIL B
canSend()	Pack 8-byte CAN frame with CRC and transmit via CAN0	Due 1 only	100 ms	FSR-13, 16, 16A	ASIL B
Serial monitor output	Print diagnostic summary every 1000 ms to Serial	Both	1000 ms	FSR-21	ASIL A

5. Software Functional Requirements

5.1 Startup and Power-On Self-Test (POST)

Before the system arms itself and enters the main monitoring loop, every safety-critical hardware resource must be verified. Any failure detected during POST results in the system remaining permanently disarmed, the deployment outputs D8 and D9 are never set HIGH until all POST checks pass.

SWR-01 POST Execution	The software shall execute <code>cpu_test()</code> , <code>ram_test()</code> , and <code>adc_test()</code> sequentially before entering the main loop. The arithmetic result of <code>cpu_test()</code> shall verify arithmetic correctness by confirming that 12345×6789 equals exactly 83,810,205. <code>ram_test()</code> shall write the pattern 0xAAAA and then 0x5555 across a 128-word buffer and verify each value. <code>adc_test()</code> shall call <code>analogRead(A0)</code> and verify the result is within valid 12-bit ADC range [0, 4095]. Results shall be stored in <code>fault_cpu</code> , <code>fault_ram</code> , and <code>fault_adc</code> boolean flags respectively.
ASIL: C	FSR trace: FSR-17

SWR-02 POST Completion Before Arming	The software shall not set any airbag output pin (D8 or D9) HIGH until all POST tests have completed and their result flags have been evaluated. If any POST flag is TRUE (fault detected), both D8 and D9 shall remain LOW for the entire current ignition cycle.
ASIL: D	FSR trace: FSR-17

SWR-03 ADC Resolution	The software shall call <code>analogReadResolution(12)</code> during <code>setup()</code> to configure the Arduino Due ADC to 12-bit mode. This produces ADC readings in the range 0–4095 corresponding to 0–3.3V.
ASIL: D	FSR trace: FSR-01, FSR-02

5.2 Sensor Reading

The three potentiometer-based sensor simulations produce analog voltages from 0V to 3.3V on ADC pins A0, A1, and A2. The software must read these values on every loop iteration, convert them to physical units, and validate their range before using them in the crash detection algorithm.

SWR-04 Sensor Acquisition Cycle	The software shall read analog pins A0 (accelerometer), A1 (gyroscope), and A2 (pressure sensor) on every main loop iteration. No blocking delay shall be used in the main loop.
ASIL: D	FSR trace: FSR-01, FSR-02

SWR-05 Physical Unit Conversion	Raw ADC readings shall be converted to physical units using the formula: $\text{physical} = (\text{raw} / 4095.0) * \text{MAX_VALUE}$, where <code>MAX_VALUE</code> is 100.0 g (accelerometer), 360.0 deg/s (gyroscope), 500.0 kPa (pressure sensor). The division operand 4095.0 shall be a floating-point literal to prevent integer division truncation. Physical values shall be stored as float variables.
--	---

ASIL: D	FSR trace: FSR-01
SWR-06 Threshold Comparison	Physical values shall be compared against: THRESH_ACCEL = 30.0 g, THRESH_GYRO = 120.0 deg/s, THRESH_PRESS = 200.0 kPa. A boolean threshold-exceeded flag shall be set for each sensor independently. These flags shall be cleared to FALSE at the start of each loop iteration before recomputation.
ASIL: D	FSR trace: FSR-03, FSR-04

5.3 Baby Seat Detection and Passenger Suppression

The passenger airbag suppression logic is an ASIL D safety function protecting against airbag deployment into a child seat. The hardware provides a 10 kΩ pull-up resistor to 3.3V on pin D11. The software implements debouncing and fail-safe default behaviour on top of this hardware measure.

SWR-07 Baby Seat Input Pin Configuration	Pin D11 (baby seat switch) shall be configured as INPUT (not INPUT_PULLUP) because an external 10 kΩ pull-up resistor to 3.3V is already provided on the hardware. Active-LOW logic applies: LOW (0V) = baby seat present, HIGH (3.3V) = adult occupant or empty seat.
ASIL: D	FSR trace: FSR-09
SWR-08 10 ms Debounce Filter	The updateBabySeat() function shall be called on every main loop iteration. A state transition shall only be accepted after the pin has remained at the new logic level for a minimum of 10 consecutive milliseconds. A timer shall restart on every pin change. Debounce time is defined by compile-time constant DEBOUNCE_MS = 10.
ASIL: D	FSR trace: FSR-11
SWR-09 Suppress-on-Fault Default	If the baby seat pin signal becomes absent, intermittent, or produces level transitions faster than the 10 ms debounce window, the software shall default babySeat = TRUE. This suppresses the passenger airbag in the absence of a verified safe signal.
ASIL: D	FSR trace: FSR-10

5.4 Crash Detection Algorithm

The crash detection algorithm fuses the three independent sensor threshold flags into a single crashLocal boolean decision. This decision is used by both boards independently to drive their respective AND gate input pins.

SWR-10 OR fusion across sensors	crashLocal shall be set TRUE if any one of accelEx, gyroEx, or pressEx is TRUE. A crash event is detected if any single sensor exceeds its threshold. This protects against a single stuck or faulty potentiometer channel triggering a spurious crash detection.
ASIL: D	FSR trace: FSR-03

SWR-11 No Blocking Code in Main Loop	No call to delay() or any other blocking construct shall appear in the main loop. All timing-based tasks (CAN TX, status print) shall use millis() comparison to ensure the loop executes as fast as possible (target < 1 ms per iteration).
ASIL: D	FSR trace: FSR-02

5.5 Deployment Output Logic

The deployment output logic evaluates the crash decision, fault state, and occupant classification simultaneously on every loop iteration. The result drives D8 and D9, which are inputs to the physical DM74LS08N AND gate. The AND gate output, and therefore the actual deployment signal is only HIGH when both boards simultaneously drive their respective pins HIGH.

SWR-12 AND Gate Input Drive Logic - Airbag 1 (Driver)	Pin D8 (Airbag 1 AND gate input) shall be set HIGH if and only if all three of the following conditions are simultaneously TRUE: (1) crashLocal is TRUE, (2) system_fault is FALSE, (3) the other board is alive. If any condition is FALSE, D8 shall be driven LOW immediately within the same loop iteration.
ASIL: D	FSR trace: FSR-03, FSR-04, FSR-08

SWR-13 Passenger Airbag Suppression - Airbag 2	Pin D9 (Airbag 2 AND gate input) shall satisfy all conditions of SWR-12 and additionally require babySeat to be FALSE. If babySeat is TRUE regardless of crash state, fault state, or other board state D9 shall be held LOW. Baby seat suppression takes unconditional priority over all other deployment conditions.
ASIL: D	FSR trace: FSR-09, FSR-10

SWR-14 Fault Inhibit — Safe State Enforcement	If system_fault is TRUE (composed of: fault_cpu OR fault_ram OR fault_adc OR fault_due1 OR fault_due2), both D8 and D9 shall immediately be driven LOW in the same loop iteration that the fault is detected. A board that detects any fault forces its AND gate inputs LOW, which collapses the AND gate output LOW regardless of the other board's state.
ASIL: D	FSR trace: FSR-20

5.6 Watchdog Supervision

The SAM3X8E hardware watchdog timer operates completely independently of software. It cannot be disabled by software once configured. Its sole purpose is to detect software lockup and force a hardware MCU reset, driving all GPIO to their default LOW state before software re-initialises.

SWR-15 Hardware Watchdog Configuration	watchdog_setup() shall configure the SAM3X8E WDT with reset-on-timeout mode (WDT_MR_WDRSTEN) and a timeout value of WDT_MR_WDV(256 * 10), corresponding to approximately 10 ms. The watchdog shall be configured during setup() before POST tests execute. Once configured, the watchdog cannot be disabled by any software instruction.
ASIL: D	FSR trace: FSR-22

SWR-16 Watchdog Feed	watchdog_feed() shall be the first function call executed in every iteration of loop(). If any loop iteration fails to call watchdog_feed() within the 10 ms timeout window, the hardware will reset the MCU to LOW, forcing AND gate inputs LOW and preventing deployment.
ASIL: D	FSR trace: FSR-22

5.7 Inter-Board CAN Communication

Both boards communicate via CAN bus. A heartbeat path detects total board failure, and a status path provides fine-grained remote state (crash, baby seat, fault).

SWR-17 CAN Status Transmission	Each board shall transmit its status string via CAN bus every 500 ms using a non-blocking millis() timer. Heartbeat interval is defined by compile-time constant HEARTBEAT_MS = 500.
ASIL: D	FSR trace: FSR-07

SWR-18 CAN Supervision Timeout	Each board shall monitor CAN bus continuously for incoming heartbeat messages from the other board. If no valid heartbeat is received for a period exceeding COMM_TIMEOUT milliseconds, the fault flag shall be set TRUE and both D8 and D9 shall be driven LOW via SWR-14.
ASIL: D	FSR trace: FSR-07, FSR-15

SWR-19 CAN Status Parsing	Due 2 shall transmit its complete status to Due 1 every 100 ms. Due 1 shall parse each received status string and extract the remote state into variables: due2_crash, due2_baby, and due2_fault.
ASIL: D	FSR trace: FSR-07

5.8 CAN Bus Communication (Due 1 Only)

Due 1 transmits system status to the OMS laboratory PC via CAN bus at 500 kbit/s. CAN communication is classified ASIL B and carries monitoring data only. Loss of CAN has no effect on the ASIL D deployment decision path.

SWR-20 CAN Initialisation	Due 1 shall initialise CAN0 at 500 kbit/s by calling Can0.begin(CAN_BPS_500K, 255) during setup(). Due 2 shall initialise CAN1 for inter-channel communication with Due 1. Due 2 does not connect to the OMS PC and shall not initialise CAN0.
ASIL: B	FSR trace: FSR-13

SWR-21 CAN Frame Structure	Every CAN frame shall use identifier 0x100 and DLC = 8 bytes with the following fixed layout: Byte 0 = flags (8 status bits), Byte 1 = Due1 accel scaled 0–255, Byte 2 = Due1 gyro scaled 0–255, Byte 3 = Due2 compact status, Byte 4 = reserved (0x00), Byte 5 = rolling sequence counter, Bytes 6–7 = CRC-16.
ASIL: B	FSR trace: FSR-13, FSR-16

SWR-22 CRC-16 Frame Protection	The crc16() function shall calculate a CRC-16 checksum using polynomial 0x1021 with initial seed value 0xFFFF over exactly the first 6 bytes (Bytes 0–5) of each CAN frame. The result shall be stored in Bytes 6 and 7 (big-endian). CRC validation shall be applied to all inter-channel CAN frames received on CAN1
ASIL: B	FSR trace: FSR-13

SWR-23 CAN Transmission Rate	Due 1 shall transmit one CAN frame every 100 ms using a non-blocking millis() timer. No delay() shall be used. The rolling sequence counter in Byte 5 shall increment by 1 on every transmitted frame and wrap from 255 back to 0.
ASIL: B	FSR trace: FSR-16

5.9 Diagnostic Warning Indicator

The DWI warning lamp (amber LED on pin D10) is the primary mechanism for alerting the driver that the airbag system is degraded. Each board independently drives its D10 output.

SWR-24 DWI Activation on Local Fault	Pin D10 (DWI warning lamp) shall be driven HIGH within one main loop cycle of any local fault flag being detected. Local fault flags are fault_cpu, fault_ram, fault_adc, fault_due1, fault_due2.
ASIL: A	FSR trace: FSR-19

SWR-25 DWI Activation on Remote Fault	Pin D10 shall also be driven HIGH if the received due2_fault flag is TRUE, even if all local fault flags are FALSE. This provides redundant warning indication regardless of which board is faulty.
ASIL: A	FSR trace: FSR-19

6. Non-Functional Software Requirements

Non-functional requirements define the performance, timing, and configuration constraints that all software modules must satisfy. These requirements are derived directly from FSR timing constraints and are verified during integration and system testing.

ID	Requirement	Value / Limit	FSR	ASIL
SWR-NF-01	Main loop cycle time -- the complete loop() execution from watchdog_feed() to return shall not exceed this limit under any operating condition	< 1 ms	FSR-02	ASIL D
SWR-NF-02	Hardware watchdog timeout -- the WDT shall be configured and shall trigger an MCU reset if loop execution exceeds this interval	10 ms	FSR-22	ASIL D

SWR-NF-03	Baby seat debounce period -- D11 must remain stable for this minimum duration before a state change is accepted	10 ms minimum stable	FSR-11	ASIL D
SWR-NF-04	CAN heartbeat transmit interval -- each board transmits its heartbeat string to the peer at this interval	500 ms	FSR-07	ASIL D
SWR-NF-05	CAN communication timeout -- absence of a valid peer heartbeat for this duration sets the CAN inter-channel fault flag	100 ms without heartbeat	FSR-15	ASIL D
SWR-NF-06	CAN frame transmit interval -- Due 1 transmits one 8-byte CAN frame to OMS PC at this rate	100 ms	FSR-16	ASIL B
SWR-NF-07	Serial diagnostic print interval -- diagnostic status summary printed to Serial at this rate	1000 ms	FSR-21	ASIL A
SWR-NF-08	ADC resolution -- the Arduino Due ADC shall be configured to 12-bit mode producing counts in the range 0 - 4095	12-bit (0–4095)	FSR-01	ASIL D
SWR-NF-09	CAN bus baud rate -- CAN0 shall be initialised at this speed	500 kbit/s	FSR-13	ASIL B

7. Software Constants and Thresholds

All safety-critical constants shall be defined as compile-time literals using `const` or `#define`. They shall not be stored as runtime variables that could be overwritten by software faults. Any change to a value in this table requires a formal change impact assessment and re-verification of the affected test cases.

Constant	Value	Unit	Meaning	FSR
ADC_MAX	4095.0	—	Arduino Due 12-bit ADC maximum count	FSR-01
ACCEL_MAX	100.0	g	Accelerometer full-scale physical range	FSR-01
GYRO_MAX	360.0	°/s	Gyroscope full-scale physical range	FSR-01
PRESS_MAX	500.0	kPa	Pressure sensor full-scale physical range	FSR-01
THRESH_ACCEL	30.0	g	Frontal crash detection threshold	FSR-03
THRESH_GYRO	120.0	°/s	Rollover detection threshold	FSR-03

THRESH_PRESS	200.0	kPa	Side impact detection threshold	FSR-06
DEBOUNCE_MS	10	ms	Baby seat pin debounce time	FSR-11
HEARTBEAT_MS	500	ms	CAN heartbeat transmit interval	FSR-07
COMM_TIMEOUT	100	ms	CAN link fault threshold	FSR-15
CAN_TX_MS	100	ms	CAN frame transmit interval	FSR-16
CAN_TX_ID	0x100	hex	CAN message identifier for OMS PC	FSR-16
CAN_BAUD	CAN_BPS_500K	—	CAN bus speed 500 kbit/s	FSR-13

Note: COMM_TIMEOUT has been updated to 100 ms from the original value of 2000 ms. A 2000 ms fault detection window is 50–100× longer than the crash event itself (20–40 ms) and is therefore not acceptable for an ASIL D system. The updated value of 100 ms ensures that a silent board is detected and flagged within one diagnostic cycle.

8. Mandatory Coding Constraints

The following coding constraints are mandatory for all ASIL D software in this system. They apply to all modules listed in Section 4. Violations must be identified and resolved before V&V testing begins.

ID	Constraint	Rationale	How to Verify
CC-01	Dynamic memory allocation (malloc, new, delete, realloc) is prohibited throughout the entire codebase	Dynamic allocation can fail at runtime in unpredictable ways and can cause memory fragmentation, leading to undefined behaviour in safety-critical code	Code review — grep for malloc, new, delete in all source files
CC-02	Recursive function calls are prohibited	Recursion depth is not statically bounded and can cause stack overflow, corrupting safety-critical variables	Code review — verify call graph has no cycles
CC-03	All global and static variables shall be explicitly initialised before first use	Uninitialised variables contain undefined values on startup, potentially causing incorrect deployment decisions	Code review + static analysis tool
CC-04	No call to delay() anywhere in the main loop	delay() blocks the entire CPU, preventing watchdog feed, sensor reads, and fault detection during the blocked period	Code review — grep for delay() in loop() scope

CC-05	All safety-critical thresholds and timing constants shall be compile-time const literals, not runtime variables	Runtime variables can be corrupted by software faults or stack overflow, changing deployment thresholds without detection	Code review — verify all THRESH_x and timing values are const
CC-06	Integer arithmetic shall not be used where floating-point precision is required in conversion formulas	Integer division of (raw / 4095) truncates to zero for all valid raw values below 4095, producing incorrect physical unit conversion	Code review — verify toPhys() uses 4095.0 not 4095
CC-07	Requirement numbers shall be referenced as comments above the implementing code block	Provides direct traceability from code to requirements, enabling efficient verification and future change impact assessment	Code review — every SWR must appear as a comment in the code
CC-08	The hardware watchdog_feed() call shall always be the first statement in loop(), with no conditional guard	Any conditional guard could allow the watchdog to expire during a fault condition, which is the correct behaviour — but if the guard prevents the feed during normal operation, it creates a false reset	Code review — verify watchdog_feed() is unconditional and first

9. Full Requirements Traceability Matrix

Every software requirement traces back to a functional safety requirement (FSR) and forward to an implementing code module and test case. This matrix ensures complete bidirectional traceability as required by ISO 26262-6 and IEC 61508-3.

SWR ID	Description	FSR	ASIL	Code Module	Test Case
SWR-01	POST execution at boot	FSR-17	ASIL C	cpu_test, ram_test, adc_test	TC-17
SWR-02	No output before POST complete	FSR-17	ASIL D	setup() sequence — POST before loop entry	TC-17
SWR-03	ADC 12-bit resolution	FSR-01, FSR-02	ASIL D	analogReadResolution(12) in setup()	TC-01
SWR-04	Sensor read every loop iteration	FSR-01, FSR-02	ASIL D	analogRead A0/A1/A2 every loop iteration	TC-01, TC-02
SWR-05	Physical conversion formula	FSR-01	ASIL D	toPhys() function	TC-01

SWR-06	Threshold comparison	FSR-03, FSR-04	ASIL D	accelEx, gyroEx, pressEx flags	TC-03, TC-04
SWR-07	Baby seat INPUT mode	FSR-09	ASIL D	pinMode(D11, INPUT) in setup()	TC-09
SWR-08	10 ms debounce filter	FSR-11	ASIL D	updateBabySeat() debounce timer	TC-11
SWR-09	Suppress-on-fault default	FSR-10	ASIL D	updateBabySeat() fault branch — babySeat=true	TC-10
SWR-10	Sensor OR fusion	FSR-03	ASIL D	crashLocal = accelEx gyroEx pressEx	TC-03
SWR-11	No blocking delay in loop	FSR-02	ASIL D	millis() timers throughout	TC-02
SWR-12	D8 AND gate drive logic	FSR-03, FSR-08	ASIL D	driveAirbag1 logic — D8 output	TC-03, TC-08
SWR-13	D9 passenger suppression	FSR-09, FSR-10	ASIL D	driveAirbag2 logic + babySeat gate	TC-09, TC-10
SWR-14	Fault inhibit both outputs	FSR-20	ASIL D	system_fault → D8/D9 LOW — enterSafeState	TC-20
SWR-15	Hardware watchdog configuration	FSR-22	ASIL D	watchdog_setup() — WDV(256×10) ≈10 ms	TC-22
SWR-16	Unconditional watchdog service call	FSR-22	ASIL D	watchdog_feed() — first call in loop()	TC-22
SWR-17	Inter-channel CAN status transmission	FSR-07	ASIL D	Channel B CAN status frame TX	TC-07
SWR-18	Inter-channel CAN supervision timeout	FSR-07, FSR-15	ASIL D	fault_peerChannel timeout detection	TC-15
SWR-19	Due 2 status TX / parseCAN	FSR-07	ASIL D	sendToMaster() / parseCAN()	TC-07
SWR-20	CAN0 init 500 kbps	FSR-13	ASIL B	Can0.begin(CAN_BPS_500K, 255)	TC-13
SWR-21	CAN 8-byte frame structure	FSR-13, FSR-16	ASIL B	canSend() frame layout	TC-13, TC-16
SWR-22	CRC-16 on CAN frame	FSR-13	ASIL B	crc16(poly=0x1021, seed=0xFFFF)	TC-13

SWR-23	CAN TX every 100 ms	FSR-16	ASIL B	millis() non-blocking CAN timer	TC-16
SWR-24	DWI on local fault	FSR-19	ASIL A	D10 HIGH on local fault flag	TC-19
SWR-25	DWI on remote fault	FSR-19	ASIL A	due2_fault → D10 HIGH	TC-19

10. Approval and Sign-Off

This document is at Revision 2.0 DRAFT status. All approvals below must be confirmed before this SRS is used as the baseline for software implementation and V&V execution.

Role	Name	Signature	Date
Software Safety Engineer		_____	_____
Hardware Safety Engineer		_____	_____
Functional Safety Engineer		_____	_____
V&V Lead		_____	_____
Project Supervisor		_____	_____